# Clase Práctica 4: Patrones de diseño orientado a objetos

Ingeniería de Software

Equipo: RAM + $L^2$

**Integrantes y partes asignadas:**

- Luis Daniel Friol Rodriguez: preguntas 2D y 4B.
- Meylí Jiménez Velázquez: preguntas 2C y restricción del ejercicio 4 (4E).
- Ailema Matos Rodriguez: preguntas 2B y 4C.
- Lianet Tamarit Tejas: preguntas 2A y 4D.
- Raúl Roberto Espinosa Poma: restricción del ejercicio 2 (2E) y pregunta 4A.

**Repositorio del proyecto:** `https://github.com/Charge1500/nutritional_management_system`

## Ejercicio 2: Sistema de Personalización de Bebidas

### Inciso a) Identificación y justificación del patrón

#### Extracción de Hechos Clave

| **Categoría** | **Hecho Clave / Requisito** |
|---|---|
| Dominio | Sistema de personalización y ventas de cafetería. |
| Bebidas Base | Café, Té, Chocolate. |
| Complementos | Leche, Azúcar, Crema, Canela, Sirope y Vainilla (esta última se incorpora en la implementación del inciso b). |
| Reglas de Negocio | ▪ Combinación libre y arbitraria de complementos.<br>▪ Cálculo dinámico del costo total acumulado. |

Cuadro 1: Matriz de hechos clave del problema.

#### Solución Ingenua (La Clase Monolítica / Anti-patrón)

Antes de aplicar patrones de diseño, una aproximación típica consistiría en crear una única clase que agrupe todas las propiedades y banderas de los complementos dentro de la misma estructura:

```
public class BebidaPersonalizada
{
    public string TipoBebida { get; set; } // "Cafe", "Te", "Chocolate"
    public int CantidadLeche { get; set; }
    public int CantidadAzucar { get; set; }
    public int CantidadCrema { get; set; }
    public int CantidadCanela { get; set; }
    public int CantidadSirope { get; set; }

    public double CalcularCosto()
```

```
    {
        double costoBase = TipoBebida == "Cafe" ? 2.00 : 1.50;
        return costoBase + (CantidadLeche * 0.50)
                         + (CantidadAzucar * 0.25)
                         + (CantidadCrema * 0.75)
                         + (CantidadCanela * 0.30)
                         + (CantidadSirope * 0.60);
    }
}
```

**¿Por qué falla este enfoque ingenuo?**

- **Violación del Principio Abierto/Cerrado (OCP):** Si el día de mañana la cafetería decide añadir un nuevo complemento (por ejemplo, *Hielo* o *Caramelo*), estás obligado a modificar el código fuente de esta clase y añadir una nueva propiedad, rompiendo el diseño existente.
- **Rigidez y Desperdicio de Memoria:** Si un cliente pide una bebida simple sin complementos, la estructura interna carga de todas formas campos e indicadores de complementos que no se están utilizando.
- **Escalabilidad nula:** Intentar soportar combinaciones complejas o múltiples porciones de forma estática sobre una sola clase genera un código lleno de condiciones difíciles de mantener.

**Análisis de Patrones Candidatos (Según Conferencia)**

| **Tipo** | **Patrón** | **Evaluación** |
|---|---|---|
| Estructural | **Decorator** | **Elegido.** Añade responsabilidades dinámicamente mediante composición en tiempo de ejecución. |
| Creacional | Factory Method | Descartado. Crea objetos base, no resuelve adiciones dinámicas en cadena. |
| Creacional | Abstract Factory | Descartado. Diseñado para familias cerradas de productos. |
| Comportamiento | Strategy | Descartado. Intercambia algoritmos excluyentes, no los acumula. |
| Estructural | Flyweight | Si, para optimización. Minimiza el uso de memoria compartiendo la mayor cantidad de datos posibles entre instancias similares. |

Cuadro 2: Evaluación de patrones candidatos de la asignatura.

**Identificación y Justificación del Patrón Decorator**

**Identificación del Patrón**

El patrón más adecuado es **Decorator (Decorador)**, de la categoría de patrones estructurales.

### Justificación Teórica

1. **Flexibilidad dinámicas:** Permite añadir complementos sobre cualquier bebida en tiempo de ejecución sin modificar las clases existentes (Principio Abierto/Cerrado).
2. **Acumulación recursiva:** Cada complemento envuelve a la bebida anterior y le suma su costo de forma transparente.
3. **Soporte de doble porción:** Permite que un complemento envuelva a otro del mismo tipo (ej. `Azucar` envuelve a `Azucar`).

### Estructura de Clases y Diagrama UML

### Clases Participantes

- `IBebida`**:** Interfaz abstracta común con métodos `GetDescripcion()` y `Costo()`.
- `Cafe`, `Te`, `Chocolate`**:** Bebidas base concretas.
- `AgregadoDecorator`**:** Decorador abstracto que implementa `IBebida` y contiene un objeto interno (`_bebida`) por composición.
- `Leche`, `Azucar`, `Crema`, `Canela`, `Sirope`, `Vainilla`**:** Decoradores concretos que añaden precio e ingrediente a la cadena.

### Diagrama de Clases UML (Texto / Estructura)

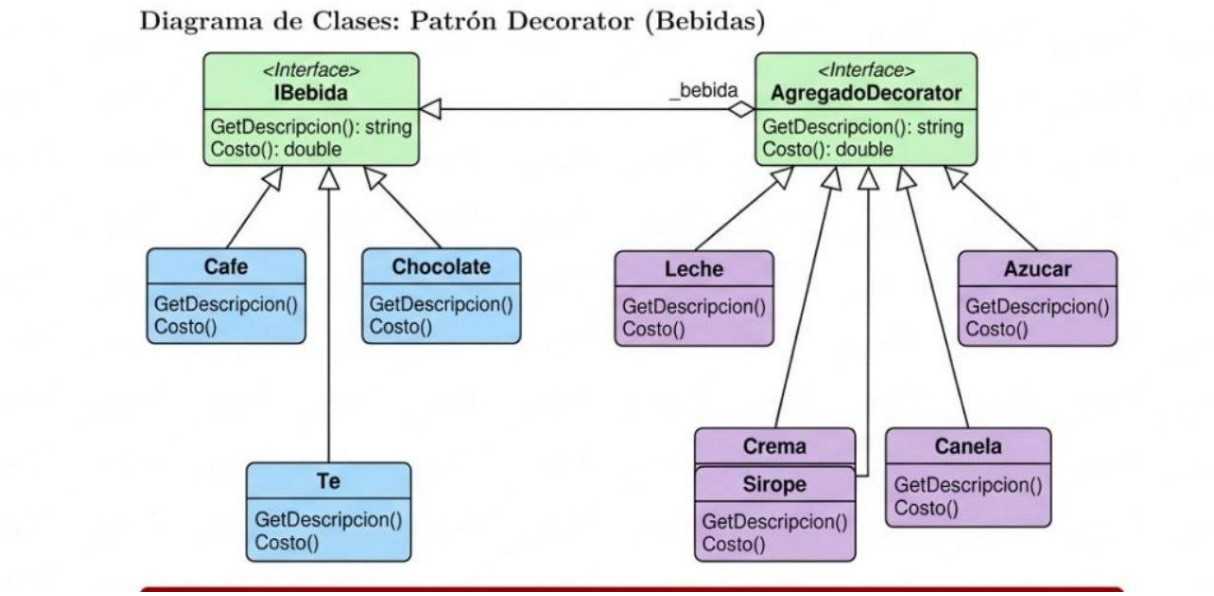



Figura 1: Diagrama de clases UML del patrón Decorator aplicado a la personalización de bebidas.

### ¿Por qué no utilizar herencia múltiple?

La herencia múltiple no es la solución más adecuada para este problema porque cada combinación de bebida y complementos podría requerir una clase específica. Por ejemplo, sería necesario representar combinaciones como `CafeConLeche`, `CafeConAzucar` o `CafeConLecheYAzucar`.

A medida que aumenta el número de complementos, la cantidad de combinaciones posibles crece considerablemente. Si se consideran seis complementos, cada uno presente o ausente, existen hasta $2^6 = 64$ combinaciones por bebida base, sin contar las cantidades múltiples de cada complemento.

Este enfoque provoca una explosión de clases y dificulta el mantenimiento y la extensión del sistema.

En cambio, Decorator permite combinar objetos mediante composición, creando las combinaciones necesarias en tiempo de ejecución sin definir una clase para cada una.

#### ¿Por qué no utilizar Strategy?

El patrón Strategy permite definir una familia de algoritmos intercambiables y seleccionar uno de ellos según las necesidades del contexto. Sin embargo, en este problema los complementos no representan algoritmos alternativos, sino características que deben acumularse sobre una bebida.

Por ejemplo, un cliente puede pedir un café con leche, azúcar y crema simultáneamente. Strategy no está diseñado para encadenar de forma natural estas responsabilidades, mientras que Decorator permite envolver sucesivamente la bebida con cada complemento.

Por tanto, Strategy podría ser útil para variar una política de cálculo de precios, pero no constituye la solución principal para representar la personalización de las bebidas.

#### Propuesta de solución y relaciones entre clases

La solución se basa en una interfaz común denominada `IBebida`, que define las operaciones `GetDescripcion()` y `Costo()`.

Las clases `Cafe`, `Te` y `Chocolate` implementan esta interfaz y representan las bebidas base. Cada una proporciona una descripción inicial y un costo base.

La clase abstracta `AgregadoDecorator` también implementa `IBebida` y mantiene una referencia a otro objeto de ese tipo. Esta referencia permite que un decorador envuelva tanto una bebida base como otro decorador.

Los decoradores concretos, como `Leche`, `Azucar`, `Crema`, `Canela`, `Sirope` y `Vainilla`, redefinen las operaciones para añadir su descripción y su costo al resultado anterior.

De esta manera, los objetos se relacionan mediante composición y todos mantienen una interfaz común. La combinación de complementos se construye dinámicamente, y un mismo complemento puede añadirse varias veces para representar porciones múltiples.

#### Ejemplo de composición

Una bebida formada por café, leche y doble azúcar puede representarse mediante la siguiente estructura:

```
Azucar( Azucar( Leche(  Cafe() ) ) )
```

Cada decorador contiene una referencia al objeto anterior. Al calcular el costo, se suma el costo base del café, el de la leche y el de las dos porciones de azúcar. La descripción se construye de manera análoga.

### Inciso b) Implementación del patrón Decorator

#### Decisiones de diseño

Se mantienen los nombres definidos en el análisis de patrones del informe: la interfaz `IBebida`, con los métodos `GetDescripcion()` y `Costo()`, y la clase abstracta `AgregadoDecorator`, que almacena una referencia a la bebida envuelta.

Las principales decisiones de diseño son:

1. **Estructura de la cadena:** el decorador base implementa `IBebida` y almacena la bebida envuelta en el atributo `_bebida`. Por defecto, delega las operaciones en ella. Cada complemento redefine `GetDescripcion()` y `Costo()` para añadir su nombre y su precio al resultado anterior.

2. **Doble porción mediante composición:** no se necesita una clase específica como `DobleAzucar`. Basta con envolver dos veces la bebida:

   ```
   new Azucar(new Azucar(bebida))
   ```

3. **Cálculo dinámico del costo:** la bebida no almacena un costo total actualizado. El costo se calcula cuando se consulta el método `Costo()`, recorriendo la cadena de decoradores.

Los precios utilizados como ejemplo se muestran en la Tabla 3.

Cuadro 3: Precios de las bebidas y los complementos

| **Producto** | **Precio** |
|---|---|
| Café | 2.00 |
| Té | 1.50 |
| Chocolate | 1.50 |
| Leche | 0.50 |
| Azúcar | 0.25 |
| Crema | 0.75 |
| Canela | 0.30 |
| Sirope | 0.60 |
| Vainilla | 0.55 |

### Interfaz de la bebida (componente abstracto)

La interfaz `IBebida` define las operaciones que deben ofrecer tanto las bebidas base como los decoradores. Esto permite tratarlos uniformemente desde el código cliente.

```
public interface IBebida
{
    string GetDescripcion();
    double Costo();
}
```

### Bebidas base (componentes concretos)

Las bebidas base implementan directamente la interfaz `IBebida`. Cada una proporciona su propia descripción y su costo inicial, sin complementos adicionales.

```
public class Cafe : IBebida
{
    public string GetDescripcion() => "Café";
    public double Costo() => 2.00;
}

public class Te : IBebida
```

```
{
    public string GetDescripcion() => "Té";
    public double Costo() => 1.50;
}

public class Chocolate : IBebida
{
    public string GetDescripcion() => "Chocolate";
    public double Costo() => 1.50;
}
```

### Decorador abstracto

La clase `AgregadoDecorator` implementa la interfaz `IBebida` y contiene una referencia a otra bebida. Su constructor comprueba que la referencia recibida no sea nula. Los métodos virtuales delegan las operaciones en el objeto envuelto y pueden ser redefinidos por los decoradores concretos.

```
public abstract class AgregadoDecorator : IBebida
{
    protected IBebida _bebida;

    protected AgregadoDecorator(IBebida bebida)
    {
        _bebida = bebida
            ?? throw new ArgumentNullException(nameof(bebida));
    }

    public virtual string GetDescripcion()
        => _bebida.GetDescripcion();

    public virtual double Costo()
        => _bebida.Costo();
}
```

### Decoradores concretos (seis complementos)

Cada complemento hereda de `AgregadoDecorator`. Su constructor recibe una bebida y la pasa al constructor de la clase base. Los métodos redefinidos añaden el nombre del complemento a la descripción y su precio al costo de la bebida envuelta.

```
public class Leche : AgregadoDecorator
{
    public Leche(IBebida bebida) : base(bebida) { }

    public override string GetDescripcion()
        => _bebida.GetDescripcion() + ", Leche";

    public override double Costo()
        => _bebida.Costo() + 0.50;
}

public class Azucar : AgregadoDecorator
```

```
{
    public Azucar(IBebida bebida) : base(bebida) { }

    public override string GetDescripcion()
        => _bebida.GetDescripcion() + ", Azúcar";

    public override double Costo()
        => _bebida.Costo() + 0.25;
}

public class Crema : AgregadoDecorator
{
    public Crema(IBebida bebida) : base(bebida) { }

    public override string GetDescripcion()
        => _bebida.GetDescripcion() + ", Crema";

    public override double Costo()
        => _bebida.Costo() + 0.75;
}

public class Canela : AgregadoDecorator
{
    public Canela(IBebida bebida) : base(bebida) { }

    public override string GetDescripcion()
        => _bebida.GetDescripcion() + ", Canela";

    public override double Costo()
        => _bebida.Costo() + 0.30;
}

public class Sirope : AgregadoDecorator
{
    public Sirope(IBebida bebida) : base(bebida) { }

    public override string GetDescripcion()
        => _bebida.GetDescripcion() + ", Sirope";

    public override double Costo()
        => _bebida.Costo() + 0.60;
}

public class Vainilla : AgregadoDecorator
{
    public Vainilla(IBebida bebida) : base(bebida) { }

    public override string GetDescripcion()
        => _bebida.GetDescripcion() + ", Vainilla";

    public override double Costo()
```

```
        => _bebida.Costo() + 0.55;
}
```

### Cliente y prueba de la solución

El cliente trabaja exclusivamente con la interfaz `IBebida`. Por tanto, no necesita distinguir si recibe una bebida simple o una cadena de decoradores. La función `Mostrar()` presenta la descripción completa y el costo con dos cifras decimales.

```
using System;
using System.Globalization;

public class Program
{
    static void Mostrar(IBebida b)
    {
        string costo = b.Costo()
            .ToString("F2", CultureInfo.InvariantCulture);

        Console.WriteLine(b.GetDescripcion() + " -> $" + costo);
    }

    public static void Main()
    {
        // 1. Bebida simple, sin complementos
        Mostrar(new Cafe());

        // 2. Café con leche y doble azúcar
        Mostrar(new Azucar(
            new Azucar(
                new Leche(new Cafe()))));

        // 3. Té con canela, sirope y vainilla
        Mostrar(new Vainilla(
            new Sirope(
                new Canela(new Te()))));

        // 4. Chocolate con crema, doble sirope y canela
        Mostrar(new Canela(
            new Sirope(
                new Sirope(
                    new Crema(new Chocolate())))));

        // 5. Café con los seis complementos
        IBebida completa = new Cafe();
        completa = new Leche(completa);
        completa = new Azucar(completa);
        completa = new Crema(completa);
        completa = new Canela(completa);
        completa = new Sirope(completa);
        completa = new Vainilla(completa);
```

```
        Mostrar(completa);
    }
}
```

La salida esperada es la siguiente:

```
Café -> $2.00
Café, Leche, Azúcar, Azúcar -> $3.00
Té, Canela, Sirope, Vainilla -> $2.95
Chocolate, Crema, Sirope, Sirope, Canela -> $3.75
Café, Leche, Azúcar, Crema, Canela, Sirope, Vainilla -> $4.95
```

La verificación del segundo caso es:

$$2{,}00 + 0{,}50 + 0{,}25 + 0{,}25 = 3{,}00$$

Para el cuarto caso:

$$1{,}50 + 0{,}75 + 0{,}60 + 0{,}60 + 0{,}30 = 3{,}75$$

En ambos casos, el costo total coincide con la suma del precio de la bebida base y los precios de todos sus complementos.

### Análisis de extensibilidad

El patrón permite incorporar nuevos productos sin modificar las clases existentes.

- **Nuevo complemento:** para añadir, por ejemplo, caramelo, basta con crear una nueva clase que herede de `AgregadoDecorator`.

  ```
  public class Caramelo : AgregadoDecorator
  {
      public Caramelo(IBebida bebida) : base(bebida) { }

      public override string GetDescripcion()
          => _bebida.GetDescripcion() + ", Caramelo";

      public override double Costo()
          => _bebida.Costo() + 0.65;
  }
  ```

- **Nueva bebida base:** para incorporar un producto como `Latte`, basta con crear una clase que implemente `IBebida`. Todos los complementos existentes podrán utilizarse con ella sin modificaciones.

En una solución monolítica, cada complemento nuevo obligaría a añadir propiedades y modificar el método que calcula el costo. Esto dificultaría el mantenimiento y contravendría el Principio Abierto/Cerrado, según el cual las clases deben estar abiertas a la extensión, pero cerradas a la modificación.

En comparación con una solución basada en herencia para cada combinación, si existen $N$ bebidas base y $M$ complementos binarios, podrían necesitarse hasta $N \cdot 2^M$ clases para representar todas las combinaciones.

Para $N = 3$ bebidas y $M = 6$ complementos:

$$N \cdot 2^M = 3 \cdot 2^6 = 192$$

Con Decorator bastan $N + M + 2$ clases: tres bebidas base, seis decoradores concretos, la interfaz y el decorador abstracto.

$$N + M + 2 = 3 + 6 + 2 = 11$$

Además, las porciones múltiples pueden representarse envolviendo repetidamente el objeto, sin crear nuevas clases específicas para cada cantidad.

### Inciso c) Descuentos por combinaciones de complementos

El patrón Decorator permite envolver un objeto con otros objetos que agregan comportamiento. En este problema, cada complemento envuelve una bebida y añade su costo al del objeto que contiene. Todos mantienen una interfaz común, por lo que pueden encadenarse. Este diseño funciona bien cuando cada complemento aporta un costo independiente. El problema surge cuando una promoción depende de la combinación de varios complementos.

La regla «Leche + Canela = 10 % de descuento» no pertenece exclusivamente a la leche ni a la canela, porque depende de una combinación presente en la bebida completa. Si se implementara directamente en las clases `Leche` o `Canela`, podrían surgir los siguientes problemas:

- **Difícil mantenimiento:** cada nueva combinación promocional obligaría a modificar clases de complementos existentes, dificultando el cumplimiento del Principio Abierto/Cerrado.
- **Promociones en conflicto:** sería necesario definir si los descuentos se acumulan, si una promoción excluye a otra y sobre qué importe se calculan.
- **Descuentos repetidos:** una misma promoción podría aplicarse varias veces si hay porciones duplicadas de leche o canela, cuando la regla debería activarse una sola vez por pedido.
- **Acoplamiento entre complementos:** la clase `Leche` tendría que conocer la existencia de `Canela` y de otras clases implicadas en promociones futuras.
- **Dependencia del orden de envoltura:** una búsqueda limitada al objeto envuelto podría detectar la combinación en `Canela(Leche(Cafe))`, pero no en `Leche(Canela(Cafe))`, aunque ambas bebidas tengan los mismos complementos.

#### Propuesta de solución

Para resolver estos problemas, se separan las responsabilidades: Decorator continúa construyendo la bebida y calculando el subtotal, mientras que las reglas de descuento se gestionan mediante un servicio de precios independiente.

Se propone utilizar los siguientes componentes:

- `DescriptorPedido`: almacena la bebida base y un diccionario de complementos con sus cantidades. El diccionario se expone mediante una interfaz de solo lectura, como `IReadOnlyDictionary`, para impedir que otros componentes modifiquen directamente los datos del pedido.
- `ConstructorBebida`: valida la bebida base, los complementos admitidos y sus cantidades positivas. Después construye la cadena de decoradores en un orden fijo de catálogo y el descriptor desde la misma receta. De esta forma, ambos representan la misma composición y la cadena no depende del orden en que llegue el pedido.

- `ServicioPrecios`: recibe el subtotal obtenido mediante `IBebida.Costo()` y el descriptor del pedido. La regla se evalúa una sola vez por pedido, independientemente del orden de los decoradores y las porciones extra.
- `IReglaDescuento`: define una interfaz común para las reglas de descuento, permitiendo incorporar nuevas promociones sin modificar el servicio de precios.
- `ReglaLecheCanela`: aplica un descuento del 10 % cuando el pedido contiene al menos una porción de leche y una de canela.

La estructura conceptual de la solución es:

```
IBebida
 |-- Cafe, Te, Chocolate
 |       (bebidas base)
 |
 |-- AgregadoDecorator
         |-- Leche
         |-- Azucar
         |-- Crema
         |-- Canela
         |-- Sirope
         |-- Vainilla

ConstructorBebida
 |-- Cadena de decoradores
 |       (calcula el subtotal)
 |
 |-- DescriptorPedido
         (complementos y cantidades)

ServicioPrecios
 |-- Recibe subtotal y descriptor
 |-- Evalúa las reglas IReglaDescuento
         |-- ReglaLecheCanela
             (Leche >= 1 y Canela >= 1
              => 10 % de descuento)
```

La regla de descuento se evalúa una sola vez por pedido, con independencia del orden de envoltura o de las porciones adicionales. Para una única promoción del 10 %, el total se calcula como:

$$\text{Total} = \text{Subtotal} - 0{,}10 \cdot \text{Subtotal}$$

si el pedido contiene al menos una porción de leche y una de canela. En caso contrario, el total coincide con el subtotal.

Si se incorporan varias promociones, debe definirse una política explícita para resolver su acumulación y evitar descuentos incompatibles. Una posibilidad consiste en calcular cada descuento sobre el subtotal original y sumar los importes resultantes.

La implementación puede verificarse con pruebas que comprueben que el descuento se aplica cuando aparece la combinación requerida, que no se aplica cuando falta alguno de los complementos y que el resultado es independiente del orden de envoltura. También debe comprobarse que una doble porción no active dos veces una promoción que solo exige la presencia de ambos complementos.

### Implementación y pruebas

El código de esta solución está en el archivo `BebidasDescuentos.cs` del repositorio. Reutiliza `IBebida` y los decoradores del inciso b). `ConstructorBebida` aplica los complementos en un orden fijo (el de su catálogo), de modo que la cadena de decoradores no depende del orden en que llegue el pedido.

```
using System;
using System.Collections.Generic;
using System.Linq;

// Receta del pedido: bebida base y cantidad de cada complemento (solo lectura)
public class DescriptorPedido
{
    public string BebidaBase { get; }
    public IReadOnlyDictionary<string, int> Complementos { get; }

    public DescriptorPedido(string bebidaBase, Dictionary<string, int> complementos)
    {
        BebidaBase = bebidaBase;
        Complementos = new Dictionary<string, int>(complementos);
    }

    public int Cantidad(string complemento)
        => Complementos.TryGetValue(complemento, out int n) ? n : 0;
}

// Construye la cadena de decoradores y el descriptor desde la misma receta
public static class ConstructorBebida
{
    private static readonly Dictionary<string, Func<IBebida>> Bases =
        new Dictionary<string, Func<IBebida>>
        {
            ["Cafe"] = () => new Cafe(),
            ["Te"] = () => new Te(),
            ["Chocolate"] = () => new Chocolate()
        };

    // Orden fijo de envoltura (determinista)
    private static readonly (string Nombre, Func<IBebida, IBebida> Decorar)[] Catalogo =
    {
        ("Leche", b => new Leche(b)),   ("Azucar", b => new Azucar(b)),
        ("Crema", b => new Crema(b)),   ("Canela", b => new Canela(b)),
        ("Sirope", b => new Sirope(b)), ("Vainilla", b => new Vainilla(b))
    };

    public static (IBebida Bebida, DescriptorPedido Descriptor) Construir(
        string bebidaBase, Dictionary<string, int> complementos)
    {
        if (!Bases.TryGetValue(bebidaBase, out var crearBase))
            throw new ArgumentException("Bebida base desconocida: " + bebidaBase);

        foreach (var par in complementos)
        {
            if (par.Value <= 0)
                throw new ArgumentException("Cantidad no positiva: " + par.Key);
            if (!Catalogo.Any(c => c.Nombre == par.Key))
                throw new ArgumentException("Complemento desconocido: " + par.Key);
        }

        IBebida bebida = crearBase();
        foreach (var (nombre, decorar) in Catalogo)
        {
            complementos.TryGetValue(nombre, out int cantidad);
            for (int i = 0; i < cantidad; i++)
                bebida = decorar(bebida);
        }
        return (bebida, new DescriptorPedido(bebidaBase, complementos));
    }
}

public interface IReglaDescuento
```

```
{
    double CalcularDescuento(DescriptorPedido pedido, double subtotal);
}

// Leche + Canela = 10 %, una sola vez por pedido (basta con >= 1 de cada una)
public class ReglaLecheCanela : IReglaDescuento
{
    public double CalcularDescuento(DescriptorPedido pedido, double subtotal)
        => pedido.Cantidad("Leche") >= 1 && pedido.Cantidad("Canela") >= 1
            ? 0.10 * subtotal : 0.0;
}

public class ServicioPrecios
{
    private readonly IEnumerable<IReglaDescuento> _reglas;

    public ServicioPrecios(IEnumerable<IReglaDescuento> reglas) { _reglas = reglas; }

    // Cada descuento se calcula sobre el subtotal original y luego se suman
    public double Total(IBebida bebida, DescriptorPedido pedido)
    {
        double subtotal = bebida.Costo();
        return subtotal - _reglas.Sum(r => r.CalcularDescuento(pedido, subtotal));
    }
}
```

Programa de prueba (en su propio proyecto o carpeta, porque define `Main`):

```
using System;
using System.Collections.Generic;
using System.Globalization;

public static class PruebaDescuentos
{
    static string F(double x) => x.ToString("F2", CultureInfo.InvariantCulture);

    static void Probar(ServicioPrecios servicio, string baseNombre,
                       Dictionary<string, int> complementos)
    {
        var (bebida, descriptor) = ConstructorBebida.Construir(baseNombre, complementos);
        Console.WriteLine(bebida.GetDescripcion() + " -> Subtotal $" + F(bebida.Costo())
                          + ", Total $" + F(servicio.Total(bebida, descriptor)));
    }

    public static void Main()
    {
        var servicio = new ServicioPrecios(new IReglaDescuento[] { new ReglaLecheCanela() });

        // 1. Leche + Canela: se aplica el 10 %
        Probar(servicio, "Cafe", new Dictionary<string, int> { ["Leche"] = 1, ["Canela"] = 1 });
        // 2. Solo leche: no hay descuento
        Probar(servicio, "Cafe", new Dictionary<string, int> { ["Leche"] = 1 });
        // 3. Mismos complementos en otro orden: mismo resultado que el caso 1
        Probar(servicio, "Cafe", new Dictionary<string, int> { ["Canela"] = 1, ["Leche"] = 1 });
        // 4. Doble porción de ambos: el descuento se aplica una sola vez
        Probar(servicio, "Cafe", new Dictionary<string, int> { ["Leche"] = 2, ["Canela"] = 2 });
    }
}
```

La salida esperada es:

```
Café, Leche, Canela -> Subtotal $2.80, Total $2.52
Café, Leche -> Subtotal $2.50, Total $2.50
Café, Leche, Canela -> Subtotal $2.80, Total $2.52
Café, Leche, Leche, Canela, Canela -> Subtotal $3.60, Total $3.24
```

Verificación del primer caso: $2{,}80 - 0{,}10 \cdot 2{,}80 = 2{,}52$ (tiene el mismo total del tercer caso aunque el pedido llegue en distinto orden). En el segundo caso se observa que sin la combinación completa no hay descuento. En el cuarto, el subtotal es $2{,}00 + 2 \cdot 0{,}50 + 2 \cdot 0{,}30 = 3{,}60$ y el

descuento es solo 0,36 (se aplica el descuento solo una vez aunque haya doble porción de ambos), por lo que el total es 3,24.

La persistencia del pedido y la reconstrucción de la bebida se desarrollan en el inciso e). El `DescriptorPedido` de este inciso cumple un papel equivalente al de la clase `Pedido` definida allí.

## Inciso d) Análisis de objetos y aplicación del patrón Flyweight

### Cantidad de objetos con Decorator tradicional

Al construir una bebida con cinco complementos (leche, azúcar doble, crema, canela y sirope), es decir, con una porción adicional de uno de ellos, se crean los siguientes objetos:

- Un objeto correspondiente a la bebida base.
- Seis objetos decoradores, uno por cada porción de complemento.

Por tanto, el total es:

$$1 + 6 = 7 \text{ objetos}$$

Si se construyen mil pedidos de esta forma, sin compartir objetos entre ellos, el número de objetos de bebida y decoradores sería:

$$1000 \cdot 7 = 7000 \text{ objetos}$$

Este número, por sí solo, no demuestra que el consumo de memoria sea excesivo: para determinarlo habría que medir el tamaño de los objetos, las referencias y la memoria total utilizada. Sin embargo, el ejemplo permite identificar una posible oportunidad de optimización en un sistema con muchos pedidos concurrentes.

### Aplicación del patrón Flyweight

Para reducir la creación repetida de objetos de complementos, se propone una alternativa basada en el patrón **Flyweight**, implementada en el archivo `BebidasFlyweight.cs`.

La solución separa el estado de cada complemento en dos partes:

- **Estado intrínseco (compartido):** contiene las propiedades que no varían entre pedidos, como el tipo de complemento y su costo unitario. Estos datos se almacenan en objetos compartidos administrados por una fábrica, denominada `FabricaComplementos`, que mantiene una caché.
- **Estado extrínseco (no compartido):** representa los datos específicos de cada uso, como la cantidad de porciones. Este estado se pasa al decorador `ComplementoFlyweight` o se mantiene en la estructura del pedido, en lugar de almacenarse en el objeto compartido.

Por ejemplo, si mil pedidos utilizan los mismos cinco tipos de complementos del ejemplo anterior, la fábrica reutiliza cinco objetos Flyweight (clase `TipoComplemento`), uno por tipo. Los pedidos mantienen sus propias cantidades y referencias, evitando crear una instancia completa del estado intrínseco para cada aparición del complemento.

En la comparación que sigue, la fábrica crea exactamente cinco objetos compartidos para los mil pedidos. La cantidad total de objetos depende además de las bebidas base y los decoradores ligeros de cada pedido.

Por tanto, la solución puede reducir el uso de memoria cuando existen muchas repeticiones de los mismos tipos de complementos. Su conveniencia debe confirmarse mediante mediciones de memoria y rendimiento bajo una carga representativa de pedidos concurrentes. La optimización no debe alterar el costo final ni permitir que el estado específico de un pedido modifique los objetos compartidos.

### Implementación y comparación de números

El código está en `BebidasFlyweight.cs`. `TipoComplemento` es el objeto compartido (estado intrínseco: nombre y precio unitario), `FabricaComplementos` lo crea una sola vez por tipo y `ComplementoFlyweight` es un decorador ligero que guarda la cantidad de porciones (estado extrínseco) y una referencia al tipo compartido.

```
using System.Collections.Generic;

public class TipoComplemento                  // Flyweight: estado intrínseco
{
    public string Nombre { get; }
    public double Precio { get; }

    public TipoComplemento(string nombre, double precio)
    {
        Nombre = nombre;
        Precio = precio;
    }
}

public static class FabricaComplementos       // caché de Flyweights
{
    private static readonly Dictionary<string, TipoComplemento> cache =
        new Dictionary<string, TipoComplemento>();

    public static int TiposCreados => cache.Count;

    public static TipoComplemento Obtener(string nombre, double precio)
    {
        if (!cache.TryGetValue(nombre, out TipoComplemento tipo))
        {
            tipo = new TipoComplemento(nombre, precio);
            cache[nombre] = tipo;
        }
        return tipo;
    }
}

// Decorador ligero: la cantidad es el estado extrínseco
public class ComplementoFlyweight : AgregadoDecorator
{
    private readonly TipoComplemento _tipo;
    private readonly int _cantidad;

    public ComplementoFlyweight(IBebida bebida, TipoComplemento tipo, int cantidad)
        : base(bebida)
    {
        _tipo = tipo;
        _cantidad = cantidad;
    }

    public override string GetDescripcion()
    {
        string d = _bebida.GetDescripcion();
        for (int i = 0; i < _cantidad; i++) d += ", " + _tipo.Nombre;
        return d;
    }

    public override double Costo() => _bebida.Costo() + _tipo.Precio * _cantidad;
}
```

Programa de comparación (en su propio proyecto o carpeta, porque define `Main`). Construye mil pedidos iguales (café con leche, azúcar doble, crema, canela y sirope) de las dos formas y cuenta los objetos creados; el conteo es por construcción, no una medición de bytes.

```
using System;
using System.Globalization;

public static class MedicionFlyweight
```

```
{
    public static void Main()
    {
        const int pedidos = 1000;
        int objetosTradicional = 0, objetosFlyweight = 0;
        IBebida ultimaT = null, ultimaF = null;

        var receta = new[] { ("Leche", 0.50, 1), ("Azúcar", 0.25, 2), ("Crema", 0.75, 1),
                             ("Canela", 0.30, 1), ("Sirope", 0.60, 1) };

        for (int i = 0; i < pedidos; i++)
        {
            // Decorator tradicional: 1 base + 1 decorador por porción = 7 objetos
            IBebida t = new Cafe();
            t = new Leche(t); t = new Azucar(t); t = new Azucar(t);
            t = new Crema(t); t = new Canela(t); t = new Sirope(t);
            objetosTradicional += 7;
            ultimaT = t;

            // Flyweight: 1 base + 1 decorador ligero por tipo = 6 objetos
            IBebida f = new Cafe();
            objetosFlyweight++;
            foreach (var (nombre, precio, cantidad) in receta)
            {
                f = new ComplementoFlyweight(f, FabricaComplementos.Obtener(nombre, precio), cantidad);
                objetosFlyweight++;
            }
            ultimaF = f;
        }
        int compartidos = FabricaComplementos.TiposCreados;
        objetosFlyweight += compartidos;

        string ct = ultimaT.Costo().ToString("F2", CultureInfo.InvariantCulture);
        string cf = ultimaF.Costo().ToString("F2", CultureInfo.InvariantCulture);

        Console.WriteLine("Pedidos construidos: " + pedidos);
        Console.WriteLine("Decorator tradicional: " + objetosTradicional + " objetos (7 por pedido)");
        Console.WriteLine("Flyweight: " + objetosFlyweight + " objetos ("
            + (objetosFlyweight - compartidos) + " en los pedidos + " + compartidos + " compartidos)");
        Console.WriteLine("Tipos compartidos creados por la fábrica: " + compartidos);
        Console.WriteLine("Costo del pedido: " + ct + " (tradicional), " + cf + " (flyweight)");
        Console.WriteLine("Misma descripción: " + (ultimaT.GetDescripcion() == ultimaF.GetDescripcion()));
    }
}
```

La salida esperada es:

```
Pedidos construidos: 1000
Decorator tradicional: 7000 objetos (7 por pedido)
Flyweight: 6005 objetos (6000 en los pedidos + 5 compartidos)
Tipos compartidos creados por la fábrica: 5
Costo del pedido: 4.65 (tradicional), 4.65 (flyweight)
Misma descripción: True
```

La comparación muestra 7000 objetos frente a 6005, es decir, 995 objetos menos (aproximadamente un 14 %), con el mismo costo y la misma descripción. La reducción en número de objetos es moderada, porque el Flyweight sigue necesitando un decorador ligero por tipo de complemento en cada pedido. Que esto se traduzca en un ahorro real de memoria sigue dependiendo de las mediciones indicadas arriba.

## Inciso e) Restricción: persistencia del pedido y reconstrucción de la bebida

**Enunciado:** el sistema debe guardar el pedido en una base de datos y poder reconstruir la bebida a partir del registro. La pregunta clave es: *¿cómo se serializa un Decorator?*

### ¿Cómo se serializa un Decorator?

**No se serializa la cadena de objetos: se guarda su receta (bebida base y lista ordenada de complementos con su cantidad) y se reconstruye con una fábrica.** Por

ejemplo, la bebida

`Canela(Azucar(Azucar(Leche(Cafe()))))`

se guarda como `Cafe` + `[Leche×1, Azucar×2, Canela×1]`. Serializar la cadena directamente no es buena idea por tres razones:

1. **Estructura anidada:** cada decorador solo guarda una referencia (`_bebida`) a lo que envuelve y no expone ninguna colección que se pueda recorrer; una base de datos relacional no la representa de forma natural.
2. **Acoplamiento al código:** guardar los objetos liga los datos a los nombres de las clases. Renombrar o refactorizar un decorador invalidaría los pedidos antiguos.
3. **El precio es derivable:** el costo se calcula al consultarlo, así que basta guardar los datos que lo determinan.

La doble porción no necesita una clase especial: se guarda `Cantidad = 2` y, al reconstruir, se envuelve la bebida dos veces con el mismo decorador. La columna `Orden` conserva el orden de envoltura, así la descripción reconstruida es idéntica a la original. Esta solución es independiente de la del inciso c): `Pedido` y `FabricaBebidas` cumplen el papel de `DescriptorPedido` y `ConstructorBebida`, pero `ConstructorBebida` aplica los complementos en un orden fijo de catálogo, mientras que aquí se conserva el orden original del pedido.

### Implementación

Se usa SQLite. La receta se guarda en dos tablas, con códigos estables (`Cafe`, `Azucar`, ...) y no las descripciones con tilde que muestra la bebida:

```
CREATE TABLE Pedido (
    Id INTEGER PRIMARY KEY AUTOINCREMENT, BebidaBase TEXT NOT NULL, Total REAL NOT NULL);
CREATE TABLE PedidoComplemento (
    PedidoId INTEGER NOT NULL, Orden INTEGER NOT NULL, Complemento TEXT NOT NULL,
    Cantidad INTEGER NOT NULL CHECK (Cantidad > 0), PRIMARY KEY (PedidoId, Orden));
```

El modelo de la receta y la fábrica que reconstruye la cadena de decoradores son (las clases `IBebida`, `AgregadoDecorator`, las bebidas y los decoradores son las del inciso b); el repositorio completo con `Guardar` y `Cargar` está en el repositorio del proyecto):

```
// ---------- Modelo: la "receta" del pedido (lo que SÍ se guarda) ----------
public class ItemComplemento
{
    public string Nombre { get; set; }
    public int Cantidad { get; set; }

    public ItemComplemento(string nombre, int cantidad)
    {
        Nombre = nombre;
        Cantidad = cantidad;
    }
}

public class Pedido
{
    public int Id { get; set; }
    public string BebidaBase { get; set; }
    public List<ItemComplemento> Complementos { get; set; } = new List<ItemComplemento>();
    public double Total { get; set; }
}

// ---------- Fábrica: reconstruye la cadena de decoradores desde la receta ----------
public static class FabricaBebidas
{
```

```
    private static readonly Dictionary<string, Func<IBebida>> bases =
        new Dictionary<string, Func<IBebida>>(StringComparer.OrdinalIgnoreCase)
        {
            ["Cafe"] = () => new Cafe(),
            ["Te"] = () => new Te(),
            ["Chocolate"] = () => new Chocolate()
        };

    private static readonly Dictionary<string, Func<IBebida, IBebida>> complementos =
        new Dictionary<string, Func<IBebida, IBebida>>(StringComparer.OrdinalIgnoreCase)
        {
            ["Leche"] = b => new Leche(b),
            ["Azucar"] = b => new Azucar(b),
            ["Crema"] = b => new Crema(b),
            ["Canela"] = b => new Canela(b),
            ["Sirope"] = b => new Sirope(b),
            ["Vainilla"] = b => new Vainilla(b)
        };

    public static IBebida Construir(Pedido pedido)
    {
        if (!bases.TryGetValue(pedido.BebidaBase, out var crearBase))
            throw new ArgumentException("Bebida base desconocida: " + pedido.BebidaBase);

        IBebida bebida = crearBase();

        foreach (ItemComplemento item in pedido.Complementos)
        {
            if (!complementos.TryGetValue(item.Nombre, out var decorar))
                throw new ArgumentException("Complemento desconocido: " + item.Nombre);

            // Doble porción = envolver N veces con el mismo decorador
            for (int i = 0; i < item.Cantidad; i++)
                bebida = decorar(bebida);
        }
        return bebida;
    }
}
```

El repositorio expone `int Guardar(Pedido p)`, que inserta la receta en una transacción, y `Pedido Cargar(int id)`, que la lee ordenada por `Orden`. Para recuperar la bebida basta `FabricaBebidas.Construir(repo.Cargar(id))`.

Al guardar y reconstruir un café con leche, doble azúcar y canela se obtiene, con los precios de la Tabla 3:

```
Guardado pedido #1: Café, Leche, Azúcar, Azúcar, Canela -> $3.30
Reconstruido pedido #1: Café, Leche, Azúcar, Azúcar, Canela -> $3.30
```

# Ejercicio 4: Sistema de Navegación GPS

## Inciso a) Identificación del patrón y diseño del sistema GPS

### Paso 0: comprender el problema

El enunciado pide una aplicación de navegación que ofrezca varios métodos para calcular rutas (más rápida, más corta, evitar autopistas, evitar peajes, ecológica, panorámica), que el usuario pueda cambiar entre ellos dinámicamente y que sea fácil añadir nuevos algoritmos de ruteo.

1. **Objetos que intervienen:** el navegador (la aplicación), los algoritmos de cálculo de ruta, los puntos de origen y destino, y la ruta resultante.
2. **Variantes:** cada criterio de ruteo es un algoritmo distinto que resuelve el mismo problema (ir de A a B) con una política diferente.
3. **Restricción principal:** el algoritmo debe poder cambiarse en tiempo de ejecución, sin reconstruir la aplicación.
4. **Necesidades futuras:** nuevos algoritmos (panorámica, evitar zonas peligrosas, etc.) e incluso cargarlos como plugins por reflexión (inciso e).

No se trata simplemente de "calcular una ruta", sino de disponer de una **familia de algoritmos intercambiables** que el cliente elige y cambia cuando lo necesita.

### Paso 1: conceptos del dominio

- **Punto:** origen, destino y puntos intermedios de la ruta.
- **Ruta:** lista de puntos, distancia en kilómetros y tiempo en minutos.
- **Algoritmo de ruteo (criterio):** Rápida, Corta, EvitarAutopistas, EvitarPeajes, Ecológica, Panorámica.
- **Navegador GPS:** la aplicación que usa el algoritmo elegido.
- **Usuario** (decide el criterio) y **tráfico** (condición externa que podría motivar un cambio).

### Paso 2: solución ingenua y sus problemas

Sin ningún patrón, lo natural es un único método con condicionales:

```
public class NavegadorGPS
{
    private string tipoRuta;

    public NavegadorGPS(string tipoRuta) { this.tipoRuta = tipoRuta; }

    public void CambiarTipo(string nuevoTipo) { tipoRuta = nuevoTipo; }

    public Ruta CalcularRuta(Punto origen, Punto destino)
    {
        if (tipoRuta == "rapida")
        {
            // Dijkstra con peso = tiempo
```

```
        }
        else if (tipoRuta == "corta")
        {
            // Dijkstra con peso = distancia
        }
        else if (tipoRuta == "sinAutopistas")
        {
            // Dijkstra descartando autopistas
        }
        else if (tipoRuta == "sinPeajes")
        {
            // Dijkstra descartando peajes
        }
        else if (tipoRuta == "ecologica")
        {
            // Dijkstra con peso = consumo estimado
        }
        else
        {
            throw new ArgumentException("Tipo de ruta desconocido");
        }
        return null;
    }
}
```

**Problemas de esta solución:**

1. **Acoplamiento fuerte:** `NavegadorGPS` contiene todos los algoritmos; una sola clase hace demasiado (clase gigante o *blob*).
2. **Crecimiento explosivo:** cada algoritmo nuevo añade otra rama al `if` y obliga a modificar una clase ya probada, lo que viola el principio abierto/cerrado.
3. **Cambio dinámico frágil:** el criterio es una cadena de texto; un error de escritura solo se detecta al ejecutar.
4. **Difícil de probar:** no se puede probar un algoritmo aislado sin pasar por el navegador completo.
5. **Imposible usar plugins:** para añadir un algoritmo hay que recompilar la clase.
6. **Poca reutilización:** otra parte de la aplicación (por ejemplo, un planificador de viajes) no puede reutilizar un algoritmo sin copiar código.

Se necesita una solución que **encapsule cada algoritmo por separado**, los haga **intercambiables en ejecución** y permita **añadir nuevos sin tocar al cliente**.

**Paso 3: patrones candidatos**

**Strategy es adecuado porque:**

- Cada criterio de ruteo se convierte en una clase independiente que implementa la misma interfaz.

| Tipo | Patrón | Evaluación |
|---|---|---|
| Comportamiento | **Strategy** | **Elegido.** Define una familia de algoritmos, encapsula cada uno y los hace intercambiables; el algoritmo varía independientemente del cliente. |
| Comportamiento | Template Method | Descartado. Fija el esqueleto del algoritmo en una clase base y las variantes se eligen por herencia, al crear el objeto. |
| Comportamiento | State | Descartado. Modela cambios de comportamiento por transiciones internas de estado, no la elección de un algoritmo. |
| Creacional | Factory Method | Parcial. Sirve para crear una estrategia, pero no para intercambiar el comportamiento del cliente. |
| Comportamiento | Command | Descartado. Encapsula peticiones (colas, deshacer), no algoritmos alternativos. |
| Estructural / Comport. | Decorator, Observer | Complementarios (inciso d): añaden restricciones sobre las estrategias o notifican cambios de ruta, pero no resuelven la intercambiabilidad. |

Cuadro 4: Evaluación de patrones candidatos para el sistema GPS.

- El cliente (el contexto) solo conoce la interfaz; no sabe si calcula la ruta más rápida o la ecológica.
- Cambiar de criterio es asignar otro objeto estrategia (composición en lugar de herencia).
- Añadir un algoritmo nuevo es añadir una clase, sin tocar el contexto ni las demás estrategias.

**Paso 4: estructura de clases**

| Rol en el patrón | Clase | Responsabilidad |
|---|---|---|
| Estrategia | `IEstrategiaRuta` | Contrato común: `Ruta Calcular(Punto origen, Punto destino)`. |
| Estrategias concretas | `EstrategiaRapida`, `EstrategiaCorta`, `EstrategiaEvitarAutopistas`, `EstrategiaEvitarPeajes`, `EstrategiaEcologica` | Cada una implementa su propio algoritmo de ruteo (simulado en el inciso b). |
| Contexto | `ContextoGPS` | Guarda una referencia a la estrategia actual, permite cambiarla y delega el cálculo en ella. |
| Datos | `Punto`, `Ruta` | Entrada y salida de las estrategias. |

Cuadro 5: Clases participantes del patrón Strategy.

**Relaciones:** el contexto mantiene una asociación con `IEstrategiaRuta` (la usa y puede reemplazarla); cada estrategia concreta realiza la interfaz; el contexto no depende de ninguna

estrategia concreta.

Contrato compartido y contexto, que se usan sin cambios en el inciso b):

```
public interface IEstrategiaRuta
{
    Ruta Calcular(Punto origen, Punto destino);
}

public class ContextoGPS
{
    private IEstrategiaRuta estrategia;

    public ContextoGPS(IEstrategiaRuta estrategiaInicial)
    {
        EstablecerEstrategia(estrategiaInicial);
    }

    public void EstablecerEstrategia(IEstrategiaRuta nueva)
    {
        if (nueva == null) throw new ArgumentNullException(nameof(nueva));
        estrategia = nueva;
    }

    public Ruta CalcularRuta(Punto origen, Punto destino)
    {
        return estrategia.Calcular(origen, destino);
    }
}
```

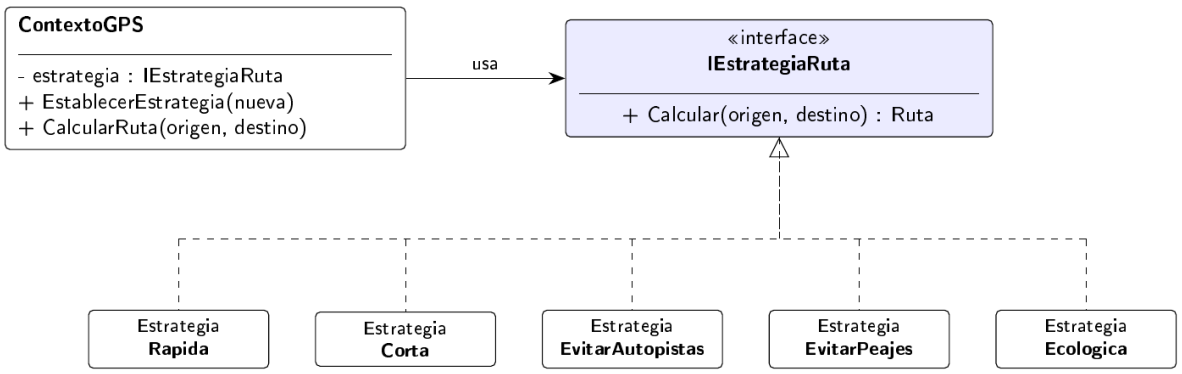



Figura 2: Diagrama de clases del patrón Strategy aplicado al sistema GPS.

### ¿Por qué no Template Method?

- **Herencia frente a composición:** Template Method define el esqueleto del algoritmo en una clase base y las subclases completan algunos pasos. La variante se fija al *crear* el objeto; para cambiar de criterio habría que crear otro objeto, lo que choca con el requisito de cambiar dinámicamente.

- **No hay un esqueleto común:** Rápida, Corta o Ecológica no comparten una secuencia fija de pasos con huecos; cambian la función de coste, el grafo que recorren o la heurística. Forzar un esqueleto común acopla las variantes a la clase base.
- **Extensión y plugins:** con Strategy cada algoritmo es un objeto independiente que puede cargarse por reflexión; con Template Method cada variante es una subclase que depende de la clase base.
- Template Method sí podría usarse *dentro* de una estrategia (por ejemplo, para compartir el recorrido de Dijkstra y variar solo el peso de cada tramo), pero no como patrón principal.

**¿Por qué no State?**

- **Intención distinta:** State permite que un objeto cambie su comportamiento cuando cambia su estado interno; los estados describen "en qué situación está el objeto". Strategy describe "cómo se realiza una tarea" y se elige desde fuera.
- **Quién decide el cambio:** en State, el contexto o los propios estados provocan las transiciones. En Strategy, el cliente (el usuario o una política externa) escoge la estrategia; las estrategias no se cambian entre sí.
- **Conocimiento mutuo:** los estados concretos suelen conocer a sus sucesores; las estrategias son independientes y no se conocen.
- **No hay ciclo de vida:** una ruta rápida no "pasa" a ser una ruta corta como consecuencia de su propia lógica; simplemente son alternativas para la misma tarea.

**Conclusión**

El patrón adecuado es **Strategy**: encapsula cada algoritmo de ruteo, permite intercambiarlos en ejecución y se extiende añadiendo clases. El contexto es `ContextoGPS`, la estrategia es `IEstrategiaRuta` y las estrategias concretas son Rápida, Corta, EvitarAutopistas, EvitarPeajes y Ecológica; la ruta panorámica del enunciado se incorpora después como extensión (inciso e).

## Inciso b) Implementación de las estrategias de ruteo

Se implementaron las cinco estrategias de ruteo requeridas: **Rápida, Corta, EvitarAutopistas, EvitarPeajes y Ecológica**, cada una como una clase que implementa la interfaz `IEstrategiaRuta`.

Cada estrategia simula su propio algoritmo de cálculo, devolviendo distancias y tiempos coherentes con su propósito. Por ejemplo, la estrategia Corta prioriza una menor distancia, aunque pueda implicar un mayor tiempo de recorrido, mientras que la estrategia Rápida prioriza la velocidad.

Además, se implementó la clase `ContextoGPS`, que permite cambiar de estrategia en tiempo de ejecución sin modificar el código cliente, cumpliendo con el principio de inversión de dependencias.

**Código**

`IEstrategiaRuta` y `ContextoGPS` son los del inciso a), sin cambios. Las clases de datos `Punto` y `Ruta` (que forman parte del contrato) y las cinco estrategias se muestran a continuación. Cada estrategia simula su algoritmo a partir de la distancia directa entre origen y destino, con un factor de recorrido y una velocidad media propios.

```
using System;
using System.Collections.Generic;
using System.Globalization;

public class Punto
{
    public string Nombre { get; }
    public double X { get; }
    public double Y { get; }

    public Punto(string nombre, double x, double y) { Nombre = nombre; X = x; Y = y; }

    public double DistanciaA(Punto otro)
        => Math.Sqrt(Math.Pow(X - otro.X, 2) + Math.Pow(Y - otro.Y, 2));
}

public class Ruta
{
    public List<Punto> Puntos { get; }
    public double DistanciaKm { get; }
    public double Minutos { get; }

    public Ruta(List<Punto> puntos, double distanciaKm, double minutos)
    {
        Puntos = puntos;
        DistanciaKm = distanciaKm;
        Minutos = minutos;
    }

    public override string ToString()
        => string.Format(CultureInfo.InvariantCulture,
            "Puntos={0}  Distancia={1:F2} km  Minutos={2:F2}", Puntos.Count, DistanciaKm, Minutos);
}

// Autopista: casi en línea recta y a 120 km/h
public class EstrategiaRapida : IEstrategiaRuta
{
    public Ruta Calcular(Punto origen, Punto destino)
    {
        double km = origen.DistanciaA(destino);
        return new Ruta(new List<Punto> { origen, destino }, km, km / 120.0 * 60.0);
    }
}

// Menor distancia (90 % de la directa), pero a 50 km/h
public class EstrategiaCorta : IEstrategiaRuta
{
    public Ruta Calcular(Punto origen, Punto destino)
    {
        double km = origen.DistanciaA(destino) * 0.90;
        return new Ruta(new List<Punto> { origen, destino }, km, km / 50.0 * 60.0);
    }
}

// Sin autopistas: pasa por una carretera nacional (+20 % de distancia, 60 km/h)
public class EstrategiaEvitarAutopistas : IEstrategiaRuta
{
    public Ruta Calcular(Punto origen, Punto destino)
    {
        double km = origen.DistanciaA(destino) * 1.20;
        var via = new Punto("Carretera-Nacional", (origen.X + destino.X) / 2, (origen.Y + destino.Y) / 2);
        return new Ruta(new List<Punto> { origen, via, destino }, km, km / 60.0 * 60.0);
    }
}

// Sin peajes: vía secundaria (+10 % de distancia, 80 km/h)
public class EstrategiaEvitarPeajes : IEstrategiaRuta
{
    public Ruta Calcular(Punto origen, Punto destino)
    {
        double km = origen.DistanciaA(destino) * 1.10;
        var via = new Punto("Via-Secundaria", (origen.X + destino.X) / 2, (origen.Y + destino.Y) / 2);
        return new Ruta(new List<Punto> { origen, via, destino }, km, km / 80.0 * 60.0);
```

```
    }
}

// Ecológica: velocidad moderada y constante (+5 % de distancia, 70 km/h)
public class EstrategiaEcologica : IEstrategiaRuta
{
    public Ruta Calcular(Punto origen, Punto destino)
    {
        double km = origen.DistanciaA(destino) * 1.05;
        var via = new Punto("Ruta-Ecologica", (origen.X + destino.X) / 2, (origen.Y + destino.Y) / 2);
        return new Ruta(new List<Punto> { origen, via, destino }, km, km / 70.0 * 60.0);
    }
}
```

Programa de prueba (en su propio proyecto o carpeta, porque define `Main`). Usa un único `ContextoGPS` y le cambia la estrategia en tiempo de ejecución:

```
using System;

public static class ProgramEstrategias
{
    public static void Main()
    {
        var origen = new Punto("A", 0, 0);
        var destino = new Punto("B", 30, 40);          // distancia directa: 50 km

        var estrategias = new (string Nombre, IEstrategiaRuta Estrategia)[]
        {
            ("rapida", new EstrategiaRapida()),
            ("corta", new EstrategiaCorta()),
            ("sinAutopistas", new EstrategiaEvitarAutopistas()),
            ("sinPeajes", new EstrategiaEvitarPeajes()),
            ("ecologica", new EstrategiaEcologica())
        };

        var contexto = new ContextoGPS(estrategias[0].Estrategia);
        foreach (var (nombre, estrategia) in estrategias)
        {
            contexto.EstablecerEstrategia(estrategia);   // cambio en tiempo de ejecución
            Console.WriteLine("[" + nombre + "] " + contexto.CalcularRuta(origen, destino));
        }
    }
}
```

La salida esperada es:

```
[rapida] Puntos=2  Distancia=50.00 km  Minutos=25.00
[corta] Puntos=2  Distancia=45.00 km  Minutos=54.00
[sinAutopistas] Puntos=3  Distancia=60.00 km  Minutos=60.00
[sinPeajes] Puntos=3  Distancia=55.00 km  Minutos=41.25
[ecologica] Puntos=3  Distancia=52.50 km  Minutos=45.00
```

La estrategia Corta recorre menos distancia (45 km) que la Rápida (50 km), pero tarda más (54 frente a 25 minutos), tal como se describió arriba.

### Análisis de extensibilidad

Gracias al patrón **Strategy** y a la carga dinámica mediante reflexión, el sistema cumple con el principio Open/Closed (abierto para extensión y cerrado para modificación).

Para añadir una sexta estrategia, por ejemplo, "Ruta con menos semáforos", se deben realizar los siguientes pasos:

1. Crear una nueva clase que implemente `IEstrategiaRuta` con el algoritmo deseado.
2. Añadir su entrada correspondiente en el archivo de configuración `estrategias.json`, indicando su nombre y tipo completo.

3. No modificar la clase `ContextoGPS`, las estrategias existentes ni el cargador por reflexión.

Esto demuestra que el sistema puede ampliarse fácilmente: el impacto de añadir una nueva estrategia se limita a incorporar una clase nueva y una entrada en el archivo JSON. El contexto y las estrategias existentes permanecen intactos, lo que facilita el mantenimiento y la evolución del sistema.

## Inciso c) Diferencia entre Strategy y State

Ambos patrones presentan una estructura similar: un contexto delega determinadas operaciones en un objeto intercambiable mediante una interfaz común. Sin embargo, se diferencian en su intención y en quién controla el cambio de comportamiento.

| **Aspecto** | **Strategy** | **State** |
|---|---|---|
| **Intención** | Encapsular algoritmos intercambiables que resuelven el mismo problema de distintas formas. | Cambiar el comportamiento de un objeto cuando cambia su estado interno. |
| **Quién decide el cambio** | El cliente o un componente externo asigna la estrategia. | Las transiciones del contexto o de los estados determinan el siguiente estado. |
| **Relación entre variantes** | Las estrategias son independientes entre sí. | Los estados pueden conocer los estados a los que pueden transitar. |
| **Ejemplo** | Calcular una ruta: rápida, corta o ecológica. | Un pedido: Creado, Pagado o Enviado. |

Cuadro 6: Comparación entre los patrones Strategy y State.

En el sistema GPS, las cinco variantes representan alternativas para realizar una misma tarea: calcular una ruta. No se transforman unas en otras ni necesitan conocerse entre sí. Por esta razón, el patrón adecuado es **Strategy**.

### ¿Puede el contexto cambiar de estrategia según el tráfico en tiempo real?

Sí. El contexto mantiene una referencia a `IEstrategiaRuta`, y no a una clase concreta, por lo que puede reasignar la estrategia durante la ejecución.

Lo importante es separar las responsabilidades. Si el contexto incorporara directamente la lógica "si hay mucho tráfico, utiliza esta estrategia", mezclaría la selección de la estrategia con su ejecución.

Una solución más adecuada consiste en separar los roles:

1. Un servicio de tráfico (`IServicioTrafico`) informa el nivel de congestión.
2. Un adaptador decide qué estrategia corresponde a cada nivel y se la asigna al contexto.
3. El contexto únicamente ejecuta la estrategia vigente.

### Ilustración en C#

Se utilizan los contratos acordados (`IEstrategiaRuta`, `Ruta` y `Punto`) y la clase `ContextoGPS` definida en el inciso 4-A y usada en el 4-B, que expone los métodos `EstablecerEstrategia()` y `CalcularRuta()`.

**Definición del servicio de tráfico y del adaptador:**

```
using System;
using System.Collections.Generic;

public enum NivelTrafico
{
    Fluido,
    Moderado,
    Intenso
}

public interface IServicioTrafico
{
    NivelTrafico ObtenerNivel(Punto origen, Punto destino);
}

// Decide que estrategia corresponde a cada nivel de trafico.
public class AdaptadorPorTrafico
{
    private readonly ContextoGPS contexto;
    private readonly IServicioTrafico trafico;
    private readonly Dictionary<NivelTrafico, IEstrategiaRuta> reglas;

    public AdaptadorPorTrafico(
        ContextoGPS contexto,
        IServicioTrafico trafico,
        Dictionary<NivelTrafico, IEstrategiaRuta> reglas)
    {
        this.contexto = contexto;
        this.trafico = trafico;
        this.reglas = reglas;
    }

    public Ruta Recalcular(Punto origen, Punto destino)
    {
        NivelTrafico nivel = trafico.ObtenerNivel(origen, destino);

        if (reglas.TryGetValue(nivel, out IEstrategiaRuta elegida))
        {
            contexto.EstablecerEstrategia(elegida);
        }

        return contexto.CalcularRuta(origen, destino);
    }
}
```

**Configuración de las reglas:**

```
var reglas = new Dictionary<NivelTrafico, IEstrategiaRuta>
{
    { NivelTrafico.Fluido, new EstrategiaRapida() },
    { NivelTrafico.Moderado, new EstrategiaEvitarAutopistas() },
    { NivelTrafico.Intenso, new EstrategiaCorta() }
```

```
};
```

**Consideraciones**

- **Extensibilidad:** cambiar la política de selección solo requiere modificar la tabla de reglas. Añadir una nueva estrategia no implica modificar el contexto.
- **Estabilidad:** si el tráfico oscila alrededor de un umbral, la ruta podría cambiar continuamente. Para evitarlo, conviene aplicar histéresis, estableciendo un tiempo mínimo o exigiendo que el cambio de tráfico se mantenga durante cierto período antes de reasignar la estrategia.
- **Aviso al usuario:** cuando la ruta cambia, el sistema puede notificarlo mediante el patrón Observer (inciso 4-D), sin que el contexto necesite conocer a los destinatarios de la notificación.

## Inciso d) Integración con Observer y Decorator

### 1. Integración con el patrón Observer

Para mantener al usuario informado sobre los cambios en el estado de la navegación sin acoplar la lógica de ruteo con la interfaz gráfica, se aplica el patrón de diseño **Observer**.

- **Sujeto (`NavegadorGPS`):** es la aplicación de navegación. En la solución final ya no contiene los algoritmos (como en la solución ingenua del inciso a)): envuelve a `ContextoGPS`, que sigue limitándose a ejecutar la estrategia vigente. El navegador mantiene la lista de observadores suscritos y los notifica cuando se recalcula una ruta o cambia la estrategia.
- **Observador (`UsuarioGPS`):** implementa `IObservadorRuta`, recibe las notificaciones del navegador y actualiza la información mostrada en la interfaz gráfica, por ejemplo, la ruta seleccionada o las instrucciones de navegación.

De esta manera, el navegador puede comunicar los cambios sin conocer los detalles de los componentes que reciben las notificaciones. Esto reduce el acoplamiento entre la lógica de navegación y la interfaz gráfica, además de facilitar la incorporación de nuevos observadores.

### 2. Integración del patrón Decorator sobre Strategy

Para añadir restricciones opcionales a las estrategias de navegación sin modificar sus implementaciones originales, se utiliza el patrón **Decorator** en combinación con el patrón **Strategy**.

- **Interfaz de estrategia (`IEstrategiaRuta`):** define la operación necesaria para calcular una ruta.
- **Decorador abstracto (`EstrategiaRutaDecorator`):** implementa `IEstrategiaRuta` y mantiene una referencia a otra estrategia de navegación, a la que delega el cálculo de la ruta.
- **Decorador concreto (`FiltroZonasPeligrosasDecorator`):** envuelve una estrategia seleccionada y añade la restricción de evitar zonas peligrosas. Para garantizar que la ruta cumpla esta condición, puede modificar los criterios de búsqueda o solicitar un nuevo cálculo que excluya dichas zonas.

Por ejemplo, se puede aplicar el decorador `FiltroZonasPeligrosasDecorator` sobre `EstrategiaRapida`, obteniendo una ruta rápida que también tenga en cuenta la restricción de seguridad, sin necesidad de modificar el código original de dicha estrategia.

Esta combinación permite separar las responsabilidades de ambos patrones: **Strategy** facilita el intercambio dinámico del algoritmo de cálculo de rutas, mientras que **Decorator** permite añadir restricciones y comportamientos adicionales de forma flexible, sin modificar las clases originales.

En conclusión, la integración de ambos patrones favorece la reutilización del código, reduce el acoplamiento y facilita la extensión del sistema de navegación.

### Código y prueba sobre `IEstrategiaRuta`

Se usan `IEstrategiaRuta`, `ContextoGPS`, `Punto`, `Ruta` y las estrategias de los incisos a) y b). Los nombres de las clases del decorador siguen el criterio de `AgregadoDecorator` (ejercicio 2): nombre en español y sufijo `Decorator`.

```
using System;
using System.Collections.Generic;
using System.Linq;

// ----- Observer -----
public interface IObservadorRuta
{
    void RutaRecalculada(string estrategia, Ruta ruta);
    void EstrategiaCambiada(string estrategia);
}

public class UsuarioGPS : IObservadorRuta
{
    private readonly string _nombre;
    public UsuarioGPS(string nombre) { _nombre = nombre; }

    public void RutaRecalculada(string estrategia, Ruta ruta)
        => Console.WriteLine("[" + _nombre + "] Ruta con " + estrategia + ": " + ruta);

    public void EstrategiaCambiada(string estrategia)
        => Console.WriteLine("[" + _nombre + "] Estrategia cambiada a " + estrategia);
}

// Sujeto: envuelve al contexto, que sigue solo ejecutando la estrategia
public class NavegadorGPS
{
    private readonly ContextoGPS _contexto;
    private readonly List<IObservadorRuta> _observadores = new List<IObservadorRuta>();
    private IEstrategiaRuta _actual;

    public NavegadorGPS(IEstrategiaRuta inicial)
    {
        _contexto = new ContextoGPS(inicial);
        _actual = inicial;
    }

    public void Suscribir(IObservadorRuta o) => _observadores.Add(o);
    public void Cancelar(IObservadorRuta o) => _observadores.Remove(o);

    public void EstablecerEstrategia(IEstrategiaRuta nueva)
    {
        _contexto.EstablecerEstrategia(nueva);
        _actual = nueva;
        foreach (var o in _observadores) o.EstrategiaCambiada(_actual.GetType().Name);
    }

    public Ruta Recalcular(Punto origen, Punto destino)
    {
        Ruta ruta = _contexto.CalcularRuta(origen, destino);
        foreach (var o in _observadores) o.RutaRecalculada(_actual.GetType().Name, ruta);
        return ruta;
    }
```

```
}

// ----- Decorator sobre Strategy -----
public abstract class EstrategiaRutaDecorator : IEstrategiaRuta
{
    protected readonly IEstrategiaRuta _interna;

    protected EstrategiaRutaDecorator(IEstrategiaRuta interna)
    {
        _interna = interna ?? throw new ArgumentNullException(nameof(interna));
    }

    public virtual Ruta Calcular(Punto origen, Punto destino)
        => _interna.Calcular(origen, destino);
}

public class FiltroZonasPeligrosasDecorator : EstrategiaRutaDecorator
{
    private readonly HashSet<string> _zonas;

    public FiltroZonasPeligrosasDecorator(IEstrategiaRuta interna, IEnumerable<string> zonas)
        : base(interna)
    {
        _zonas = new HashSet<string>(zonas);
    }

    public override Ruta Calcular(Punto origen, Punto destino)
    {
        Ruta ruta = _interna.Calcular(origen, destino);
        var seguros = ruta.Puntos.Where(p => !_zonas.Contains(p.Nombre)).ToList();
        if (seguros.Count == ruta.Puntos.Count) return ruta;

        // Simulación del nuevo cálculo sin esas zonas: 10 % más larga y lenta
        return new Ruta(seguros, ruta.DistanciaKm * 1.10, ruta.Minutos * 1.10);
    }
}
```

Programa de prueba (en su propio proyecto o carpeta, porque define `Main`). Comprueba ambos patrones: el usuario recibe los avisos del navegador y la estrategia `EvitarAutopistas`, decorada, evita la zona peligrosa sin modificarse.

```
using System;

public static class PruebaObserverDecorator
{
    public static void Main()
    {
        var a = new Punto("A", 0, 0);
        var b = new Punto("B", 30, 40);
        var zonas = new[] { "Carretera-Nacional" };

        var navegador = new NavegadorGPS(new EstrategiaRapida());
        navegador.Suscribir(new UsuarioGPS("Conductor"));

        navegador.Recalcular(a, b);
        navegador.EstablecerEstrategia(
            new FiltroZonasPeligrosasDecorator(new EstrategiaEvitarAutopistas(), zonas));
        navegador.Recalcular(a, b);

        Console.WriteLine("Sin decorador: " + new EstrategiaEvitarAutopistas().Calcular(a, b));
    }
}
```

La salida esperada es:

```
[Conductor] Ruta con EstrategiaRapida: Puntos=2  Distancia=50.00 km  Minutos=25.00
[Conductor] Estrategia cambiada a FiltroZonasPeligrosasDecorator
[Conductor] Ruta con FiltroZonasPeligrosasDecorator: Puntos=2  Distancia=66.00 km  Minutos=66.00
Sin decorador: Puntos=3  Distancia=60.00 km  Minutos=60.00
```

La última línea muestra que `EstrategiaEvitarAutopistas` pasa por `Carretera-Nacional` (3 puntos); con el decorador la ruta ya no la incluye (2 puntos) y el cálculo se hace sobre la misma interfaz, sin tocar la clase original.

## Inciso e) Carga de estrategias como plugins en tiempo de ejecución

**Restricción:** las estrategias deben poder cargarse como plugins en tiempo de ejecución mediante reflexión.

### Estructura de la solución

La solución se organizó en tres proyectos independientes, ubicados en el repositorio en la ruta `Assignments/CP4/Problem-4`:

```
GPS-Plugins/
|
+-- Contratos/                  # Contrato compartido
|   +-- Contratos.cs            # IEstrategiaRuta, Ruta, Punto
|   +-- Contratos.csproj
|
+-- NavegadorApp/               # Aplicacion base
|   +-- CargadorEstrategias.cs  # Carga por reflexion
|   +-- EstrategiaRapida.cs
|   +-- Program.cs              # No conoce estrategias concretas
|   +-- estrategias.json        # Configuracion
|   +-- NavegadorApp.csproj     # Referencia solo a Contratos
|
+-- PluginTuristica/            # Plugin adicional
    +-- EstrategiaTuristicaPanoramica.cs
    +-- PluginTuristica.csproj  # Referencia solo a Contratos
```

La separación en tres proyectos permite que `NavegadorApp` no tenga una referencia directa al proyecto `PluginTuristica`. Por tanto, el plugin puede incorporarse posteriormente sin necesidad de recompilar la aplicación base.

El proyecto `Contratos` contiene las definiciones compartidas por la aplicación y los plugins. El cargador comprueba que cada tipo encontrado implemente la interfaz `IEstrategiaRuta` mediante:

```
typeof(IEstrategiaRuta).IsAssignableFrom(tipo)
```

Esta comprobación garantiza que el plugin respete el contrato esperado antes de intentar instanciarlo.

### Configuración de las estrategias

El archivo `estrategias.json` declara las estrategias disponibles. Para cada una se especifican su nombre y su tipo completo. En el caso de los plugins externos, también se indica el nombre de su DLL. El archivo mostrado es el final; al inicio solo contiene la entrada `rapida`, y la de `turistica` se añade en el paso 3 de la demostración.

```
{
  "estrategias": [
    {
```

```
      "nombre": "rapida",
      "tipoCompleto": "NavegadorApp.EstrategiaRapida, NavegadorApp"
    },
    {
      "nombre": "turistica",
      "tipoCompleto":
        "PluginTuristica.EstrategiaTuristicaPanoramica, PluginTuristica",
      "dll": "PluginTuristica.dll"
    }
  ]
}
```

### Cargador por reflexión y plugin

El cargador lee `estrategias.json`; si la entrada trae una DLL, la carga antes de resolver el tipo con `Type.GetType`, comprueba que implemente `IEstrategiaRuta` y lo instancia con `Activator.CreateInstance`:

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Reflection;
using System.Text.Json;

public static class CargadorEstrategias
{
    public static Dictionary<string, IEstrategiaRuta> Cargar(string rutaJson)
    {
        var estrategias = new Dictionary<string, IEstrategiaRuta>();
        using JsonDocument doc = JsonDocument.Parse(File.ReadAllText(rutaJson));

        foreach (JsonElement entrada in doc.RootElement.GetProperty("estrategias").EnumerateArray())
        {
            string nombre = entrada.GetProperty("nombre").GetString();
            string tipoCompleto = entrada.GetProperty("tipoCompleto").GetString();

            // Plugin externo: se carga su DLL antes de resolver el tipo
            if (entrada.TryGetProperty("dll", out JsonElement dll))
                Assembly.LoadFrom(Path.Combine(AppContext.BaseDirectory, dll.GetString()));

            Type tipo = Type.GetType(tipoCompleto);
            if (tipo == null || !typeof(IEstrategiaRuta).IsAssignableFrom(tipo))
            {
                Console.WriteLine("[Cargador] '" + nombre + "' ignorada: tipo no válido.");
                continue;
            }

            estrategias[nombre] = (IEstrategiaRuta)Activator.CreateInstance(tipo);
            Console.WriteLine("[Cargador] Estrategia '" + nombre + "' cargada:\n"
                + tipo.FullName + " desde " + tipo.Assembly.GetName().Name + "\n");
        }
        return estrategias;
    }
}
```

La estrategia del plugin está en `PluginTuristica`, que solo referencia a `Contratos`. Pasa por tres puntos de interés entre origen y destino:

```
using System.Collections.Generic;

namespace PluginTuristica
{
    public class EstrategiaTuristicaPanoramica : IEstrategiaRuta
    {
        public Ruta Calcular(Punto origen, Punto destino)
        {
```

```
            double km = origen.DistanciaA(destino) * 1.375;   // desvíos panorámicos
            var puntos = new List<Punto>
            {
                origen,
                Intermedio(origen, destino, 0.25, "Mirador"),
                Intermedio(origen, destino, 0.50, "Parque"),
                Intermedio(origen, destino, 0.75, "Playa"),
                destino
            };
            return new Ruta(puntos, km, km / 60.0 * 60.0);     // a 60 km/h
        }

        private static Punto Intermedio(Punto o, Punto d, double t, string nombre)
            => new Punto(nombre, o.X + (d.X - o.X) * t, o.Y + (d.Y - o.Y) * t);
    }
}
```

### Demostración de la carga dinámica

Para demostrar que el contexto y la aplicación base no necesitan recompilarse al incorporar un nuevo plugin, se siguieron los siguientes pasos.

**1. Compilación y ejecución inicial de la aplicación base** Primero se compila y ejecuta únicamente la aplicación base, utilizando la estrategia rápida (el `estrategias.json` inicial solo contiene `rapida`), y se guarda el hash SHA-256 del ejecutable:

```
cd GPS-Plugins/NavegadorApp
dotnet run
sha256sum bin/Debug/net8.0/NavegadorApp.dll     # HASH_BASE
```

**2. Compilación independiente del plugin** A continuación, se compila el proyecto del plugin turístico:

```
cd ../PluginTuristica
dotnet build
```

**3. Incorporación del plugin** Se copia la DLL generada a la carpeta de salida de la aplicación:

```
cp bin/Debug/net8.0/PluginTuristica.dll \
   ../NavegadorApp/bin/Debug/net8.0/
```

También se edita una sola vez el archivo `estrategias.json` de la carpeta de salida, añadiendo la entrada `turistica` mostrada arriba.

**4. Carga de las estrategias sin recompilar** Se ejecuta directamente el binario ya compilado (no `dotnet run`, que volvería a compilar), y el cargador detecta tanto la estrategia original como el plugin externo:

```
cd ../NavegadorApp
dotnet bin/Debug/net8.0/NavegadorApp.dll
```

La salida obtenida fue:

```
[Cargador] Estrategia 'rapida' cargada:
NavegadorApp.EstrategiaRapida desde NavegadorApp

[Cargador] Estrategia 'turistica' cargada:
```

```
PluginTuristica.EstrategiaTuristicaPanoramica desde PluginTuristica

Estrategias disponibles: rapida, turistica

[rapida]    Puntos=2  Distancia=50.00 km  Minutos=25.00
[turistica] Puntos=5  Distancia=68.75 km  Minutos=68.75
```

**5. Verificación de que la aplicación base no fue recompilada** Se calcula de nuevo el hash SHA-256 del ejecutable y se compara con el del paso 1:

```
sha256sum bin/Debug/net8.0/NavegadorApp.dll    # HASH_TRAS_PLUGIN
```

Los resultados fueron:

```
HASH_BASE:         914f55a13ee295ea945ef39e21d5eb87b2873066cb148ad7829ce654d28a073f

HASH_TRAS_PLUGIN:  914f55a13ee295ea945ef39e21d5eb87b2873066cb148ad7829ce654d28a073f
```

Ambos valores coinciden, lo que demuestra que el ejecutable base permaneció sin cambios durante la incorporación del plugin.

### Conclusión

La implementación demuestra que es posible extender el sistema GPS mediante plugins cargados dinámicamente con reflexión. La aplicación base depende únicamente del contrato común y no de las implementaciones concretas de las estrategias. También, ejecuta las estrategias cargadas a través de ContextoGPS (contexto definido en el inciso a), sin conocer ninguna estrategia concreta.

De esta manera, se puede incorporar una nueva estrategia sin modificar ni recompilar la aplicación principal, siempre que el plugin respete la interfaz definida y se encuentre correctamente configurado.